

Experiment.9

Aim: To implement and demonstrate the working of Service Worker events like fetch, sync, and push in an E-commerce Progressive Web Application (PWA).

Theory:

Service Worker: A Service Worker is a script that runs in the background of a browser independently of a web page. It acts as a programmable network proxy on the client side and is designed to enhance the capabilities of web apps by enabling features like offline support, background sync, and push notifications.

Key features of a Service Worker:

- Service workers intercept and handle network requests, allowing for smart caching and background content fetching.
- They only work over HTTPS for security reasons.
- Service workers become idle when not in use and restart when needed.
- They are heavily based on Promises and asynchronous event handling.

Fetch Event: The fetch event enables the service worker to intercept network requests made by the application. There are typically two strategies used:

1. Cache First:

- Check if the request exists in the cache.
- If available, return the cached response.
- Otherwise, fetch from the network.

2. Network First:

- Attempt to fetch from the network first.
- If successful, update the cache with the new response.
- If the network request fails, return the cached response if available.

Sync Event (Background Sync): Background Sync allows tasks like data submission to be deferred until the device has a stable internet connection. It is

useful in scenarios like email sending or form submission in poor network conditions.

Workflow:

1. User initiates an action (e.g., form submission) when offline.
2. Data is stored locally (e.g., in IndexedDB).
3. The service worker registers a sync event.
4. When online, the service worker performs the action (e.g., sending data to the server).

Push Event: The push event is triggered when a push notification is sent from the server to the client. Service workers can handle these notifications and display them even if the web app is not currently open.

Workflow:

1. The server sends a push message to the service worker.
2. The service worker receives the push event and displays a notification.
3. The user can interact with the notification, which can trigger further actions.

Code :

Service Worker.js

```
const CACHE_NAME = 'custom-tee-v1';
const FILES_TO_CACHE = [
  '/',
  '/index.html',
  '/cart.html',
  '/createForm.html',
  '/shop.html',
  '/shop2.html',
  '/offline.html',
  '/styles.css',
  '/cart.css',
  '/shop.css',
  '/script.js',
```

```
    '/manifest.json',  
    '/tee-1.png',  
    '/tee-2.png',  
    '/home-img.png',  
    '/shop-img.png',  
    '/image.png',  
    '/image1.png',  
    '/image2.png',  
    '/image3.png'  
  ];  
};
```

```
// Install Service Worker
```

```
self.addEventListener('install', (event) => {  
  console.log('[Service Worker] Install');  
  event.waitUntil(  
    caches.open(CACHE_NAME).then((cache) => {  
      console.log('[Service Worker] Caching files');  
      return cache.addAll(FILEES_TO_CACHE);  
    })  
  );  
  self.skipWaiting();  
});
```

```
// Activate Service Worker
```

```
self.addEventListener('activate', (event) => {  
  console.log('[Service Worker] Activate');  
  event.waitUntil(  
    caches.keys().then((keyList) =>  
      Promise.all(  
        keyList.map((key) => {  
          if (key !== CACHE_NAME) {  
            console.log('[Service Worker] Removing old cache', key);  
            return caches.delete(key);  
          }  
        })  
      )  
    )  
  )  
});
```

```
);  
self.clients.claim();  
});
```

```
// Fetch
```

```
self.addEventListener('fetch', (event) => {  
  console.log('[Service Worker] Fetch', event.request.url);  
  event.respondWith(  
    fetch(event.request)  
      .then((response) => {  
        return caches.open(CACHE_NAME).then((cache) => {  
          cache.put(event.request, response.clone());  
          return response;  
        });  
      })  
      .catch(() => {  
        return caches.match(event.request).then((response) => {  
          return response || caches.match('/offline.html');  
        });  
      })  
  );  
});
```

```
// SYNC
```

```
self.addEventListener('sync', function(event) {  
  console.log('🔄 Sync event triggered:', event.tag);  
  if (event.tag === 'custom-tees-sync') {  
    event.waitUntil(  
      Promise.resolve().then(() => {  
        console.log('✅ Sync successful');  
      })  
    );  
  }  
});
```

Bestsellers
Men
Women
Kids &
Babies
Accessories
Gifts
Collections

CustomTee

Find designs or products

CREATE

Personalized products
for any event or occasion

Start Designing Now

SHOP

Unique designs on any
product by
independent designers

Something

Application

Manifest

Service workers

Storage

Local storage

Session storage

Extension storage

IndexedDB

Cookies

Private state tokens

Interest groups

Shared storage

Cache storage

Storage buckets

Background services

Back/forward cache

Background fetch

Background sync

Bounce tracking miti...

Notifications

Payment handler

http://127.0.0.1:5500/

Source service-worker.js

Received 4/7/2025, 2:34:17 PM

Status #900 activated and is running Stop

Clients http://127.0.0.1:5500/

Push { "method": "pushMessage", "message": "Hello from P... } Push

Sync hellosync Sync

Periodic sync test-tag-from-devtools Periodic sync

Update Cycle

Version Update Activity Timeline

#900 Install

#900 Wait

#900 Activate

Service workers from other origins

See all registrations

Console

AI assistance

Issues

[Service Worker] Fetching: http://127.0.0.1:5500/manifest.json service-worker.js:65

[Service Worker] Serving from cache: http://127.0.0.1:5500/styles.css service-worker.js:69

[Service Worker] Serving from cache: http://127.0.0.1:5500/ service-worker.js:69

[Service Worker] Serving from cache: http://127.0.0.1:5500/manifest.json service-worker.js:69

Notification displayed! service-worker.js:115

Bestsellers
Men
Women
Kids &
Babies
Accessories
Gifts
Collections

CustomTee

Find designs or products

CREATE

Personalized products
for any event or occasion

Start Designing Now

SHOP

Unique designs on any
product by
independent designers

Something

Application

Manifest

Service workers

Storage

Local storage

Session storage

Extension storage

IndexedDB

Cookies

Private state tokens

Interest groups

Shared storage

Cache storage

Storage buckets

Background services

Back/forward cache

Background fetch

Background sync

Bounce tracking miti...

Notifications

Payment handler

Periodic background ...

Speculative loads

Timestamp Event Origin Storage Key Service ... Instance ID

1 2025-04-07 15:44... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

2 2025-04-07 15:45... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

3 2025-04-07 15:45... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

4 2025-04-07 16:16... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

5 2025-04-07 16:16... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

6 2025-04-07 16:17... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

7 2025-04-07 16:17... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

8 2025-04-07 16:26... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

9 2025-04-07 16:26... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:26... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:26... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:31... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:32... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:32... Notification displayed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:36... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

1. 2025-04-07 16:36... Notification closed http://127.0.0.1:5500/ http://127.0.0.1:5500/ /

No event selected

Select an event to view its metadata

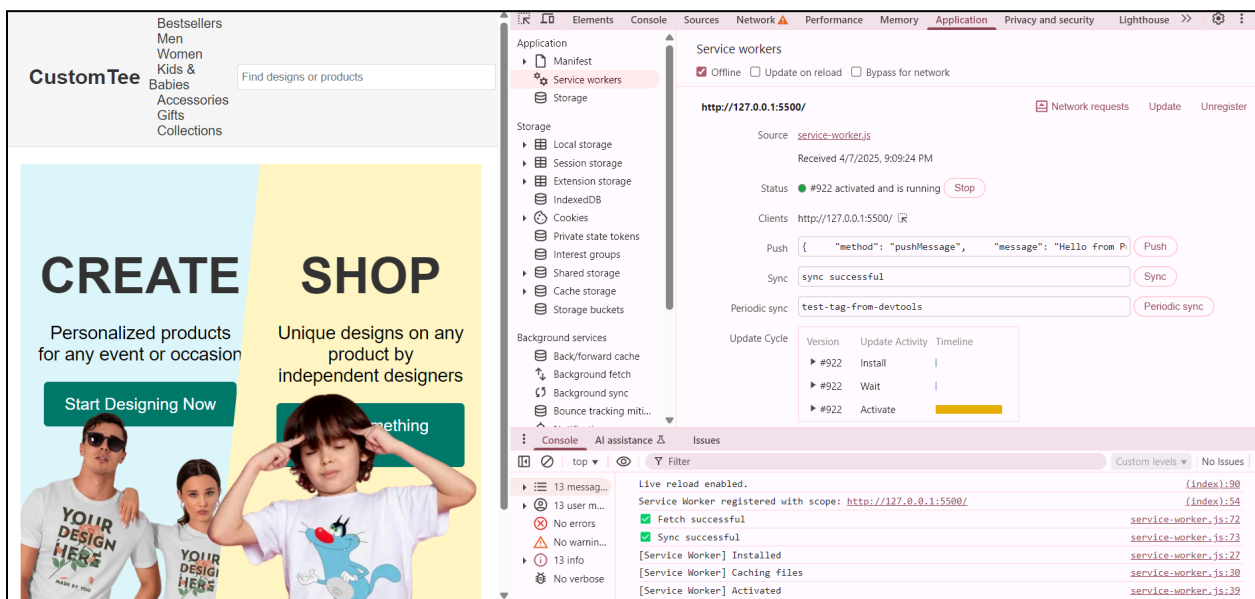
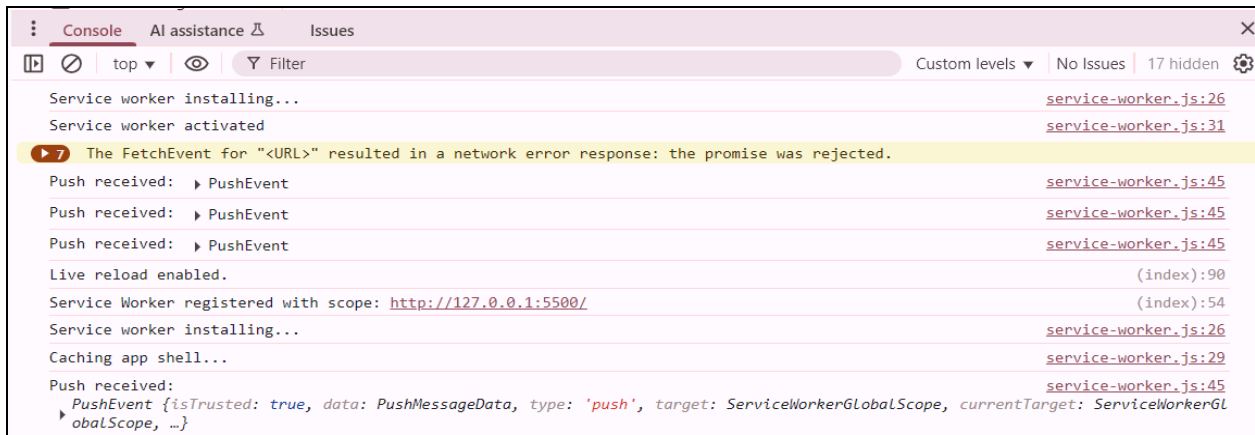
Console

AI assistance

Issues

Push received: service-worker.js:65

PushEvent {isTrusted: true, data: PushMessageData, type: 'push', target: ServiceWorkerGlobalScope, currentTarget: ServiceWorkerGlobalScope, ...}



Conclusion: Service workers significantly enhance the functionality and reliability of web applications. With events like fetch, sync, and push, developers can provide offline capabilities, delay tasks until connectivity is restored, and engage users with timely notifications. These features make PWAs more robust and user-friendly, especially in low or intermittent network environments.